

✓ Lab 5 Report:

Create Arthur Conan Doyle AI with RNN

✓ Nate Chaoweeraprasit

```
1  %matplotlib inline
2
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import torch
6  from torch.distributions import Categorical
```

```
1  from google.colab import drive
2  drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount

```
1  from IPython.display import Image # For displaying images in colab jupyter c
```

✓ Prepare Data

```
1 #device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
2 device = torch.device("cuda")
3 print(device)
4
5 # You will train on the first N characters of the Sherlock Holmes book
6 # Pick the size of your training data, i.e. N
7 data_size_to_train = 50000
8
9 # Load the Sherlock Holmes data up to data_size_to_train
10 data = open('/content/drive/MyDrive/phys 417/lab_05/sherlock.txt', 'r').read
    [:data_size_to_train]
11
12 # Find the set of unique characters within the training data
13 characters = sorted(list(set(data)))
14
15 # total number of characters in the training data and number of unique
    characters
16 data_size, vocab_size = len(data), len(characters)
17
18 print("Data has {} characters, {} unique".format(data_size, vocab_size))
```

```
cuda
Data has 50000 characters, 73 unique
```

```
1 # Use Python Dictionary to map the characters to numbers and vice versa
2 # one hot?
3 char_to_num = { ch:i for i,ch in enumerate(characters) }
4 num_to_char = { i:ch for i,ch in enumerate(characters) }
```

```
1 # Use the character_to_num dictionary to map each character in the training d
2 data = list(data)
3
4 for i, ch in enumerate(data):
5     data[i] = char_to_num[ch]
6
7 print(data[:10])
```

```
[0, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

✓ Define Model

```
1 class CharRNN(torch.nn.Module):
2
3     def __init__(self, num_embeddings, embedding_dim, input_size, hidden_size
4
5         super(CharRNN, self).__init__()
6
7         # map char integers to vectors
8         self.embedding = torch.nn.Embedding(num_embeddings, embedding_dim)
9
10        # set up core RNN layer with relu activation
11        self.rnn = torch.nn.RNN(input_size=input_size, hidden_size=hidden_size,
12                                num_layers=num_layers,
13                                nonlinearity = 'relu')
14
15        # remap hidden state
16        self.decoder = torch.nn.Linear(hidden_size, output_size)
17
18    def forward(self, input_seq, hidden_state):
19
20        # step input char index through embedding layer
21        embedding = self.embedding(input_seq)
22
23        # step embeddings + hidden state through RNN
24        output, hidden_state = self.rnn(embedding, hidden_state)
25
26        # convert output into character scores (to eval which char to print n
27        output = self.decoder(output)
28
29        # return output char prediction + hidden state
30        return output, hidden_state.detach()
```

✓ Define Hyperparameters

```
1 # Fix random seed
2 torch.manual_seed(25)
3
4 # Define RNN network
5 rnn = CharRNN(num_embeddings = vocab_size,
6               embedding_dim = 128,
7               input_size = 128,
8               hidden_size = 512,
9               num_layers = 2,
10              output_size=vocab_size)
11
12 # Define learning rate and epochs
13 learning_rate = 0.001
14 epochs = 100
15
16 # Size of the input sequence to be used during training and validation
17 training_sequence_len = 100
18 validation_sequence_len = 250
19
20 # Define loss function and optimizer
21 loss_fn = torch.nn.CrossEntropyLoss()
22 optimizer = torch.optim.Adam(rnn.parameters(), lr=learning_rate)
23
24 # add .cuda() for GPU acceleration
25 rnn = rnn.to(device)
```

✓ Identify Tracked Values

```
1 # Tracking training loss per each input/target sequence fwd/bwd pass
2 training_loss = []
```

✓ Train Model

```
1 data = torch.unsqueeze(torch.tensor(data), dim = 1)
2 data = data.to(device)
3
4 # Training Loop -----
5
6 for epoch in range(epochs):
7
8     # fetch char from first 100 chars to start
9     character_loc = np.random.randint(100)
10
```

```

11     iteration = 0
12     hidden_state = None
13
14     # Sample and generate a text sequence after every epoch -----
15
16     while character_loc + training_sequence_len + 1 < data_size:
17
18         # input = char sequence
19         # target = incremented index char sequence (shifted 1)
20         input_seq = data[character_loc : character_loc + training_sequence_len]
21         target_seq = data[character_loc + 1 : character_loc + training_sequence_len + 1]
22
23         input_seq = input_seq.to(device)
24         target_seq = target_seq.to(device)
25
26         # forward pass
27         output, hidden_state = rnn(input_seq, hidden_state)
28
29         # evaluate loss + format to tensor w/ squeeze
30         loss = loss_fn(torch.squeeze(output), torch.squeeze(target_seq))
31
32         # store loss value for data vis
33         training_loss.append(loss.item())
34
35         # backpropagation process
36         optimizer.zero_grad()
37         loss.backward()
38         optimizer.step()
39
40         # update starting character for next sequence
41         character_loc += training_sequence_len
42
43         # iterate iterations
44         iteration += 1
45
46     print(f"EPOCH {epoch+1:02d} | AVERAGE TRAINING LOSS: {np.mean(training_loss):.4f}")
47
48     character_loc = 0
49     hidden_state = None
50
51     # start from random char to init process
52     rand_index = np.random.randint(data_size-1)
53     input_seq = data[rand_index : rand_index+1]
54
55
56     print("-----")
57
58     while character_loc < validation_sequence_len:
59
60         # forward pass to predict next char off of current sequence

```

```

61         output, hidden_state = rnn(input_seq, hidden_state)
62
63         # softmax score into prob dist
64         # take dist and sample value to get character + print
65         output = torch.nn.functional.softmax(torch.squeeze(output), dim=0)
66         character_distribution = torch.distributions.Categorical(output)
67         character_num = character_distribution.sample()
68
69         print(num_to_char[character_num.item()], end='')
70         # pass char back into model to repeat for net iterations
71         input_seq[0][0] = character_num.item()
72
73         # iterate char location index
74         character_loc += 1
75
76     print("\n-----\n")

```

EPOCH 01 | AVERAGE TRAINING LOSS: 2.197404

oritg

exsty, think Shere quade the lostiont innorsarn all, stir in the pan
and atcerthen had the pire of the ome sto, stofmed are mes herk is a

EPOCH 02 | AVERAGE TRAINING LOSS: 1.805861

ide of the wright minduritazoly as mory upons overyeet. I carder the
rehick the smock and ablatenter, bestery which me the ding stave

EPOCH 03 | AVERAGE TRAINING LOSS: 1.630256

tI kind mart heart, will sackniebs or.a shoulder walls and higdeasiti
the hur your this the passment was could alary great the
the itsissed the restrest folide,

EPOCH 04 | AVERAGE TRAINING LOSS: 1.495451

arden here in anylecs with ngrey. The could he was of not coloured by
"That's "There wall, I said, Here flaring a quitred to the which bea
And
murileable of eyes or drem, what to be at blold case met of

```
-----
EPOCH 05 | AVERAGE TRAINING LOSS: 1.376418
-----
```

```
Hols at theorish
the
happy placely. "Nothing, bothere's
nargen upon the varpermariably, I casteness," he said Holmes."
```

```
"Why, stops?"
```

```
"We could not many, but him to leard was second will not me drally
was ut the remlock Holmes of the burrea
-----
```

```
-----
EPOCH 06 | AVERAGE TRAINING LOSS: 1.277084
-----
```

```
have from meting round I have a
friend
he was duning such asceaths sealution
a dken sprtiation was stalws which from the bushled upon the whole a
```

```
At the room, how Letwrenged furrier orself-and hy still when and rem
-----
```

```
-----
EPOCH 07 | AVERAGE TRAINING LOSS: 1.192502
-----
```

```
has planes."
```

✓ Visualize & Evaluate Model

```
1 # Print a validation text sequence that most closely resembles Sherlock Holme
```

ent watcoloured vacant sinaly. "The Book of Life," which was intense, learing his of a gentleman, with who candually began to everything, which is a puzzler. If you had drawn Two door were man, fot his perceptive faculties, the busgled on an infur

```

1 # Import seaborn for prettier plot
2 import seaborn as sns
3
4 #sns.set(style = 'whitegrid', font_scale = 2.5)

```

```

1 # Plot the training loss and rolling mean training loss with respect to
  iterations
2 # Feel free to change the window size
3 plt.figure(figsize = (32, 20))
4
5 plt.plot(training_loss, linewidth = 3, label = 'training loss')
6 plt.plot(np.convolve(training_loss, np.ones(100), 'valid') / 100,
7           linewidth = 3, label = 'training loss rolling avg')
8 plt.ylabel("training loss")
9 plt.xlabel("iterations")
10 plt.grid(alpha = 0.5)
11 plt.legend()
12 plt.title("training loss by iteration")

```

Text(0.5, 1.0, 'training loss by iteration')



